

QUESTIONS 2 / 2 completed

Sahred

SAHRED

Write a Java program to demonstrate how to use synchronization to prevent multiple threads from accessing a shared resource simultaneously.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         SharedResource shared = new SharedResource();
4         Thread t1 = new WorkerThread(shared, "Thread-1");
5         Thread t2 = new WorkerThread(shared, "Thread-2");
6         t1.start();
7         t2.start();
8     }
9 }
10 class SharedResource {
11     public synchronized void useResource(String threadName) {
12         System.out.println(threadName + " entered critical section.");
13         try {
14             Thread.sleep(1000);
15         } catch (InterruptedException e) {
16             e.printStackTrace();
17         }
18         System.out.println(threadName + " leaving critical section.");
19     }
20 }
21 class WorkerThread extends Thread {
```

INPUT

Your INPUT goes
here!

OUTPUT

```
Thread-1 entered critical section.
Thread-1 leaving critical section.
Thread-2 entered critical section.
Thread-2 leaving critical section.
```



SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS 2 / 2 completed

PROGRAM EDITOR

Java

Run

Save

INPUT

AtomicInteger

ATOMICINTEGER

Write a Java program to demonstrate how to use the AtomicInteger class to provide atomic access to a shared integer variable.

Output:
Shared value: 20000

This confirms that the two threads were able to increment the

```
1 import java.util.concurrent.atomic.AtomicInteger;
2 public class Main {
3     private static AtomicInteger sharedValue = new AtomicInteger(0);
4     public static void main(String[] args) throws InterruptedException {
5         int numberOfThreads = 2;
6         int incrementsPerThread = 10000;
7         Thread[] threads = new Thread[numberOfThreads];
8         for (int i = 0; i < numberOfThreads; i++) {
9             threads[i] = new Thread(() -> {
10                 for (int j = 0; j < incrementsPerThread; j++) {
11                     sharedValue.incrementAndGet();
12                 }
13             });
14             threads[i].start();
15         }
16         for (Thread t : threads) {
17             t.join();
18         }
19         System.out.println("Shared value: " + sharedValue.get());
20     }
21 }
```

Your INPUT goes here!

OUTPUT

```
Shared value: 20000
```